

CPT102 Data Structures

Week 3 practice exercises

Erick Purwanto, Thomas Selig, Kok Hoe Wong

March 30, 2026

Exercise 1 Warm-up: missing methods

In the lecture, we saw pseudocode implementations of `get(index)`, `add(element)` and `insert(index, element)`¹ for the `ArrayList` and `LinkedList` data structures. In this exercise, you will implement two additional List operations. First, recall the data models of `ArrayList` and `LinkedList`.

<code>ArrayList<E>:</code>	<code>LinkedList<E>:</code>
<code> E[] elements;</code>	<code> Node<E> head;</code>
<code> int size</code>	<code> Node<E> tail;</code>
	<code> int size;</code>
	<code>Node<E>:</code>
	<code> E data;</code>
	<code> Node<E> next;</code>

* 1. (*List searching*)

In pseudocode, write an algorithm `CONTAINS(list, element)` that returns **true** if **list** contains the specified **element**, and **false** otherwise, for both the `ArrayList` and `LinkedList` data structure models above. What is the time and space complexity in both cases?

2. (*Removal by index*)

** a) In pseudocode, write an algorithm `REMOVE(list, index)` that removes the element in **list** at the specified **index**, for `ArrayList` and `LinkedList`. What is the time and space complexity in both cases?

** b) Now write the same algorithm for the *doubly-linked list* case (see Lecture notes, slide 28). How does this affect the time complexity?

** 3. (*Coding practice*)

Repeat the above two questions in code rather than pseudocode by completing the bodies of the `contains()` and `remove()` methods in the skeletal `ArrayList` and `LinkedList` classes provided on Learning Mall here. You may also write methods corresponding to some of the algorithms seen in the lecture.

¹This is written as `add(index, element)` in Java.

Answer of exercise 1

1. (*List searching*)

For ArrayList:

```
Procedure CONTAINS(list, element)
foreach item in list.elements do
    | if item = element then
    |     | return true;
return false;
```

For LinkedList:

```
Procedure CONTAINS(list, element)
current ← list.head;
while current ≠ null do
    | if current.data = element then
    |     | return true;
    | current ← current.next;
return false;
```

Complexity: both algorithms need to traverse the list, and require no additional memory usage (other than simple variables). The complexities are therefore $O(n)$ in time (linear) and $O(1)$ in space (constant) in both cases.

2. (*Removal by index*)

a) For ArrayList:

```
Procedure REMOVE(list, index)
for i ← index to list.size - 1 do
    | list.elements[i] ← list.elements[i + 1];
    | ▷ assume that list.elements[n] = null
list.size ← list.size - 1;
```

Time complexity = $O(n - k)$, space complexity = $O(1)$.

For LinkedList:

```
Procedure REMOVE(list, index)
if index = 0 then
    | list.head ← list.head.next;
else
    | i ← 0; previous ← list.head;
    | while i < index - 1 do
    |     | previous ← previous.next; i++;
    |     | ▷ traverse to previous = list.get(index - 1)
    | previous.next ← previous.next.next;
list.size ← list.size - 1;
```

Time complexity = $O(k)$, space complexity = $O(1)$.

- b) For doubly-linked lists, recall that each node stores pointers to both its predecessor and successor in the list, i.e.:

```
Node<E>:
    E data;
    Node<E> prev;
    Node<E> next;
```

Note that the actual `LinkedList<E>` class model is unchanged, only `Node<E>` changes. Allowing iteration to start from the tail of the list where more efficient gives the following algorithm for removal.

Procedure REMOVE(list, index)

```
 $n \leftarrow \text{list.size};$ 
 $\triangleright$  cases  $\text{index} = 0$  and  $\text{index} = \text{list.size} - 1$  should be handled separately
if  $\text{index} < n/2$  then
     $i \leftarrow 0$ ;  $\text{previous} \leftarrow \text{list.head};$ 
    while  $i < \text{index} - 1$  do
         $\text{previous} \leftarrow \text{previous.next}; i++;$ 
         $\triangleright$  traverse to  $\text{previous} = \text{list.get}(\text{index} - 1)$ 
     $\text{successor} \leftarrow \text{previous.next.next};$   $\triangleright$  new successor of previous
     $\text{previous.next} \leftarrow \text{successor};$ 
     $\text{successor.prev} \leftarrow \text{previous};$ 
else
     $i \leftarrow n - 1$ ;  $\text{successor} \leftarrow \text{list.tail};$ 
    while  $i > \text{index} + 1$  do
         $\text{successor} \leftarrow \text{successor.prev}; i--;$ 
         $\triangleright$  traverse to  $\text{successor} = \text{list.get}(\text{index} + 1)$ 
     $\text{previous} = \text{successor.prev.prev};$ 
     $\text{successor.prev} = \text{previous};$ 
     $\text{previous.next} = \text{successor};$ 
 $\text{list.size} \leftarrow \text{list.size} - 1;$ 
```

The space complexity remains constant. The time complexity improves to $O(m)$, where $m = \min\{k, n - k\}$.

3. (Coding practice)

The full codes for sample `ArrayList` and `LinkedList` classes can be found in the Learning Mall folder here.

Exercise 2 Queue implementations

In this exercise, We will look into two realisations of the *Queue* abstract data type (ADT) and its standard specified operations. Recall that a Queue is a linear ADT where new elements are added to the *tail* via `enqueue(element)`, and removed from the *head* via `dequeue()`. A Queue also has a `peek()` operation to inspect its head element, and a `size()` operation to determine its number of elements.

* 1. (*Data model*)

In this exercise, we will consider two realisations of Queue based on an array or a linkedlist. We will call these **ArrayQueue** and **LinkedListQueue** respectively. The **ArrayQueue** will be a resizable-array model, with free space at both ends of the array. The **LinkedListQueue** model will be a singly-linked list. Write pseudocode for the two data models, using the lecture notes for inspiration if needed. Note that the **ArrayQueue** model will need to store the position of both the head and tail elements in the queue.

** 2. (*LinkedQueue* operations)

In pseudocode, implement the standard operations for the `LinkedList` data structure. Unlike in the lecture, you should also consider the case where the queue is empty.

3. (*ArrayQueue* operations)

We will now work on implementing the standard operations for `ArrayQueue`. Recall that the data model is:

ArrayQueue<E>:

```
E[] elements;
```

```
int head; // index of element at the head of the queue in the array
```

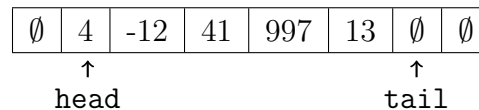
```
int tail; // index of first empty space at the tail of the queue
```

** a) Write a helper procedure `RESIZE(queue)` which doubles the *capacity* of the queue (the length of its array of elements). That is, the procedure copies all the elements of the array `queue.elements` into a new array `newElements`. This array will have twice the length of the original array, with the indices of original elements unchanged. The `RESIZE(queue)` procedure then sets its array of elements as this new array. Figure 1 illustrates this `RESIZE()` operation.

The array

```
queue.elements
```

before resizing:



The array

```
queue.elements
```

after resizing:

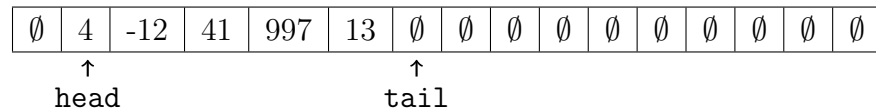


Figure 1: Illustrating the `RESIZE()` operation.

- * b) Now implement the standard queue operations for the **ArrayQueue** data structure in pseudocode. The **RESIZE()** operation from the previous question should be called when there is no remaining space at the tail of the queue.
4. (*CircularArrayQueue implementation*)
- a) Suppose that at some point in a program's execution, data is being stored in an **ArrayQueue**, with given **head** and **tail** indices. Now assume that we enqueue a new element to the tail of the queue, and then dequeue the element at its head.
- ** i – How does this sequence of operations affect the queue's **head** and **tail** indices? How does it effect the *size* of the queue?
- *** ii – Why is this problematic in terms of how often the **RESIZE()** operation may need to be invoked?
- *** b) The solution to this issue is to use a **circular** variant of **ArrayQueue**, called **CircularArrayQueue**. In this implementation, when the **tail** pointer reaches the back of the **elements** array, it “wraps around” back to the front. This is illustrated in Figure 2 below. In pseudocode, implement all four standard

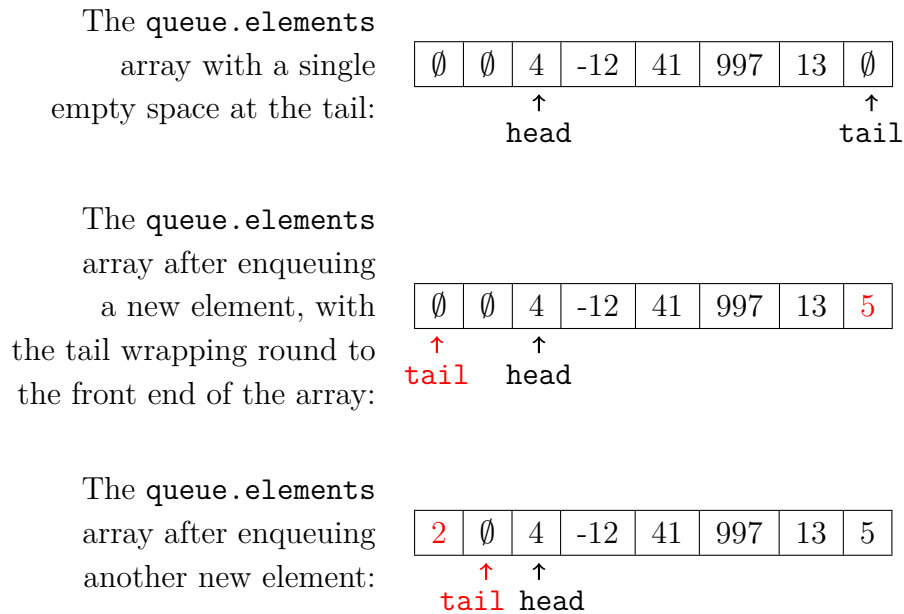


Figure 2: Illustrating the **CircularArrayQueue** structure.

queue operations on this new **CircularArrayQueue** structure. Here the **RESIZE()** operation should be called when there is no empty space left in the **elements** array. Note that, while repeated enqueueing may cause the **tail** pointer to wrap around, as in Figure 2, repeated dequeuing may in turn cause the **head** pointer to wrap around.

Answer of exercise 2

1. (*Data model*)

```
ArrayQueue<E>:
    E[] elements;
    int head;
    int tail;
```

The ArrayQueue class satisfies that $\text{elements}[i] \neq \text{null}$ if and only if $\text{head} \leq i < \text{tail}$, i.e. tail is the index of the first empty space at the end of the array.

```
LinkedListQueue<E>:
    Node<E> head;
    Node<E> tail;
    int size;
```

```
Node<E>:
    E data;
    Node<E> next;
```

2. (*LinkedListQueue operations*)

We write **FAIL** to indicate that an operation is invalid on a specific input.

Algorithm 1: Standard operations on LinkedListQueue

```
Procedure ENQUEUE(queue, element)
    newTailNode  $\leftarrow$  new Node(element, null);
    queue.tail.next  $\leftarrow$  newTailNode;
    queue.tail  $\leftarrow$  newTailNode;
    queue.size  $\leftarrow$  queue.size + 1;
```

```
Procedure PEEK(queue)
    if queue.head = null then
        FAIL;
    return queue.head.data;
```

```
Procedure DEQUEUE(queue)
    if queue.head = null then
        FAIL;
    temp  $\leftarrow$  queue.head;
    queue.head  $\leftarrow$  queue.head.next;
    queue.size  $\leftarrow$  queue.size - 1;
    return temp.data;
```

3. (*ArrayQueue operations*)

- a) The resize procedure can be written as follows. This assumes that by default, all entries in an array are set to null.

```
Procedure RESIZE(queue)
    newElements  $\leftarrow$  new E[2 * queue.elements.length];
    for i  $\leftarrow$  head to tail - 1 do
        newElements[i]  $\leftarrow$  queue.elements[i];
    queue.elements  $\leftarrow$  newElements;
```

- b) As in the previous question, we write **FAIL** to indicate that an operation is invalid on a specific input.

Algorithm 2: Standard operations on ArrayQueue

Procedure SIZE(queue)

return queue.tail – queue.head;

Procedure ENQUEUE(queue, element)

if queue.tail = queue.elements.length **then**

 RESIZE(queue);

queue.elements[tail] ← element;

queue.tail ← queue.tail + 1;

Procedure PEEK(queue)

if queue.head = queue.tail **then**

FAIL;

▷ cannot peek into an empty queue

return queue.elements[head];

Procedure DEQUEUE(queue)

if queue.head = queue.tail **then**

FAIL;

temp ← queue.elements[head];

queue.elements[head] ← null;

queue.head ← queue.head + 1;

return temp;

4. (*CircularArrayQueue* implementation)

- a) i – Enqueuing an element increments queue.tail and increases the size by one, while dequeuing increments queue.head and decreases the size by one. As such, the sequence enqueue() + dequeue() has no effect on the size, but shifts both head and tail pointers in the array by one spot (towards the tail end of the array).
- ii – When a program uses a queue ADT, elements are typically enqueued and dequeued at the same rate. If we interpret this as a randomised model over time where, at each step we choose to enqueue or dequeue at random (with probability 1/2 for each operation), then after n steps there will be roughly $n/2$ shifts of the head and tail pointer. One can also show that the size of the queue itself (its number of elements) will be of order $\sqrt{n} \ll n$. This implies that the “empty space” at the head of the queue will be of order n , followed by a significantly smaller number (order $\sqrt{n}$) of non-empty spaces. As such, a large amount of memory is being wasted. Moreover, there will be a need for frequent resizing due to the tail pointer hitting the end of the array, which is also wasteful (since there is “space” at the head of the array). Resizing a queue requires copying all its elements, which is costly in both time and space. This explains why the approach above is problematic.

- b) The data model here is unchanged, so we just write the standard operations.

Algorithm 3: Standard operations on ArrayCircularQueue

Procedure SIZE(queue)

```
return queue.tail - queue.head;
```

Procedure ENQUEUE(queue, element)

if `queue.tail = queue.head` and `queue.elements[head] ≠ null` then

```
RESIZE(queue);
```

```
queue.elements[tail]  $\leftarrow$  element;
```

```
queue.tail ← queue.tail + 1;
```

if `queue.tail = queue.elements.size` then

```
queue.tail ← 0;
```

Procedure PEEK(queue)

if `queue.elements[head] = null` then

```

|      FAIL;

```

- ▷ cannot peek into an empty queue

```
return queue.elements[head];
```

Procedure DEQUEUE(queue)

if `queue.elements[head] = null` then

```

|      FAIL;

```

```
temp ← queue.elements[head];
```

```
queue.elements[head]  $\leftarrow$  null;
```

```
queue.head  $\leftarrow$  queue.head + 1;
```

```
if queue.head == queue.elements.size then
```

```
queue.head  $\leftarrow$  0;
```

```
return temp;
```

Note that the `RESIZE()` operation is slightly different for the circular model. We resize when the array is full, i.e. the tail and head point to the same array index. In this case, we double the size of the array, with the additional empty spaces being placed between the tail and head, as on Figure 3 below.

The array

```
queue.elements
```

before resizing:

17	4	-12	41	997	13	0	-2
----	---	-----	----	-----	----	---	----

↑
tail = head

The array

```
queue.elements
```

after resizing:

17	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	4	-12	41	997	13	0	-2
----	-------------	-------------	-------------	-------------	-------------	-------------	-------------	-------------	---	-----	----	-----	----	---	----

↑
tail

↑
head

Figure 3: Illustrating the `RESIZE()` operation on a circular queue.

▷ This procedure is invoked only when we enqueue a new element and we have
`queue.tail = queue.head`

Procedure RESIZE(*queue*)

```

n ← queue.elements.length;
queue.head ← queue.head + n;
newElements ← new E[2 * n];
for i ← 0 to queue.tail - 1 do
    | newElements[i] ← queue.elements[i];
for i ← queue.tail to queue.head - 1 do
    | newElements[i] ← null;
    | ▷ unnecessary if null is the default array value, but included here to make
    |   process more explicit
for i ← queue.head to n - 1 do
    | newElements[i] ← queue.elements[i - n];
queue.elements ← newElements;

```